

Puntatori

Puntatori

Il concetto di puntatori in C++, come funzionano e come usarli.

Cos'è un puntatore?

Immagina di avere un quaderno con le istruzioni per trovare un tesoro. Il quaderno non contiene il tesoro — contiene l'**indirizzo** dove trovarlo.

Un **puntatore** funziona allo stesso modo: non contiene un valore direttamente, ma contiene la **posizione in memoria** dove quel valore si trova.

Questo ti permette di fare cose che altrimenti non potresti fare, come modificare una variabile dall'interno di una funzione.

La memoria del computer come una via di appartamenti

Pensa alla RAM come a una lunghissima via con migliaia di appartamenti. Ogni appartamento ha un **numero civico** (l'indirizzo) e può contenere un valore.

Quando dichiari `int x = 42;`, il computer affitta un appartamento per `x` e ci mette dentro il numero 42.

Un puntatore è come un bigliettino su cui scrivi il numero civico di quell'appartamento. Con il bigliettino, puoi andare a vedere cosa c'è dentro — o addirittura cambiarlo.

Come vedere l'indirizzo di una variabile

Usa l'operatore `&` davanti a una variabile per leggere il suo indirizzo:

```
int x = 42;
cout << x << endl;    // stampa 42 (il valore)
cout << &x << endl;   // stampa qualcosa come 0x7fff5c1a2b40 (l'indirizzo)
```

Quell'indirizzo strano con lettere e numeri è scritto in **esadecimale** — è il numero del "cassetto" nella RAM.

Dichiarare un puntatore

Si usa il simbolo `*` dopo il tipo:

```
int* ptr;          // puntatore a int (un bigliettino con l'indirizzo di un
intero)
double* dptr;      // puntatore a double
char* cptr;        // puntatore a char
```

Assegnare un indirizzo al puntatore

Per far "puntare" il puntatore a una variabile, assegnagli il suo indirizzo con `&` :

```
int x = 42;
int* ptr = &x;    // ptr contiene l'indirizzo di x (il "numero civico" di
x)

cout << ptr << endl;    // stampa l'indirizzo di x
cout << *ptr << endl;   // stampa 42 (il valore dentro quell'indirizzo)
```

Leggere e modificare il valore: l'operatore `*`

Mettere `*` davanti a un puntatore significa "vai all'indirizzo e leggi (o scrivi) il valore lì dentro". Si chiama **dereferenziazione**.

```
int x = 42;
int* ptr = &x;

cout << *ptr << endl;    // legge il valore di x → stampa 42

*ptr = 100;               // scrive un nuovo valore nella posizione di x
cout << x << endl;       // x è cambiata! stampa 100
```

È come andare all'appartamento indicato sul bigliettino e sostituire il contenuto.

Riepilogo dei simboli

```
int x = 42;
int* ptr = &x;

// & davanti a una variabile = "dammi l'indirizzo di questa variabile"
cout << &x << endl;    // indirizzo di x

// * nella dichiarazione = "questa variabile è un puntatore"
int* ptr = &x;          // ptr è un puntatore a int

// * davanti a un puntatore = "vai all'indirizzo e leggi/scrivi il valore"
cout << *ptr << endl;  // il valore a quell'indirizzo (42)
```

Il puntatore nullo

Se un puntatore non deve puntare a niente, assegnagli `nullptr` :

```
int* ptr = nullptr;    // non punta a nessun indirizzo

// Prima di usare un puntatore, controlla che non sia nullo!
if (ptr != nullptr) {
    cout << *ptr;    // sicuro solo se ptr non è nullptr
}
```

Regola d'oro: non usare mai un puntatore nullo senza averlo controllato prima — causerebbe il crash del programma.

Passare variabili alle funzioni tramite puntatore

Il caso d'uso più utile dei puntatori è permettere a una funzione di modificare una variabile che si trova fuori da essa:

```

// La funzione riceve il "bigliettino" con l'indirizzo di x
void raddoppia(int* ptr) {
    *ptr = *ptr * 2;    // va all'indirizzo e modifica il valore lì
}

int main() {
    int x = 5;
    cout << x << endl;    // 5

    raddoppia(&x);        // passa l'indirizzo di x (il "bigliettino")

    cout << x << endl;    // 10 - x è stata modificata dalla funzione!
    return 0;
}

```

Senza i puntatori, la funzione riceverebbe solo una **copia** di `x` e le modifiche andrebbero perse.

Puntatori e array

Il nome di un array è già un puntatore al suo primo elemento. Puoi muoverti tra gli elementi sommando numeri all'indirizzo:

```

int numeri[] = {10, 20, 30, 40, 50};
int* ptr = numeri;    // punta al primo elemento (numeri[0])

cout << *ptr << endl;    // 10 (primo elemento)
cout << *(ptr + 1) << endl;    // 20 (secondo elemento)
cout << *(ptr + 2) << endl;    // 30 (terzo elemento)

// Equivalente all'accesso normale con []
cout << ptr[0] << endl;    // 10
cout << ptr[1] << endl;    // 20

```

Esempio pratico: scambiare due variabili

```
#include <iostream>
using namespace std;

// Riceve i bigliettini con gli indirizzi di a e b
void scambia(int* a, int* b) {
    int temp = *a;    // salva il valore di a in temp
    *a = *b;          // copia il valore di b in a
    *b = temp;        // copia temp (vecchio a) in b
}

int main() {
    int x = 10, y = 20;
    cout << "Prima: x=" << x << ", y=" << y << endl;

    scambia(&x, &y);    // passa gli indirizzi di x e y

    cout << "Dopo:  x=" << x << ", y=" << y << endl;
    return 0;
}
```

Output:

```
Prima: x=10, y=20
Dopo:  x=20, y=10
```

Errori comuni da evitare

```
int* ptr;        // NON inizializzato: contiene un indirizzo casuale,
                 // pericoloso!
*ptr = 42;       // ERRORRE: modifica una zona di memoria a caso → crash

int* ptr2 = nullptr;
*ptr2 = 42;      // ERRORRE: dereferenziare nullptr → crash garantito
```

In C++ moderno, si preferisce usare i **riferimenti** quando possibile (vedi la sezione successiva) e i **puntatori smart** per la memoria dinamica.